PROJECT DOCUMENTATION

REVOLUTIONIZING AGRICULTURE WITH AGRI EDGE

PROJECT OVERVIEW AND OBJECTIVES

PROJECT OVERVIEW

Agri Edge Order Management CRM is a customized Salesforce application designed to streamline agricultural supply chain sales, order management, payment verification, and shipment tracking.

The application centralizes order processing and reduces manual work through Salesforce automation and Apex programming.

The system provides automated follow-up task creation, order total calculation, payment status tracking, shipment record creation, and order item management.

It helps businesses improve operational efficiency, maintain data accuracy, and manage the complete order fulfillment process.

PROJECT OBJECTIVES

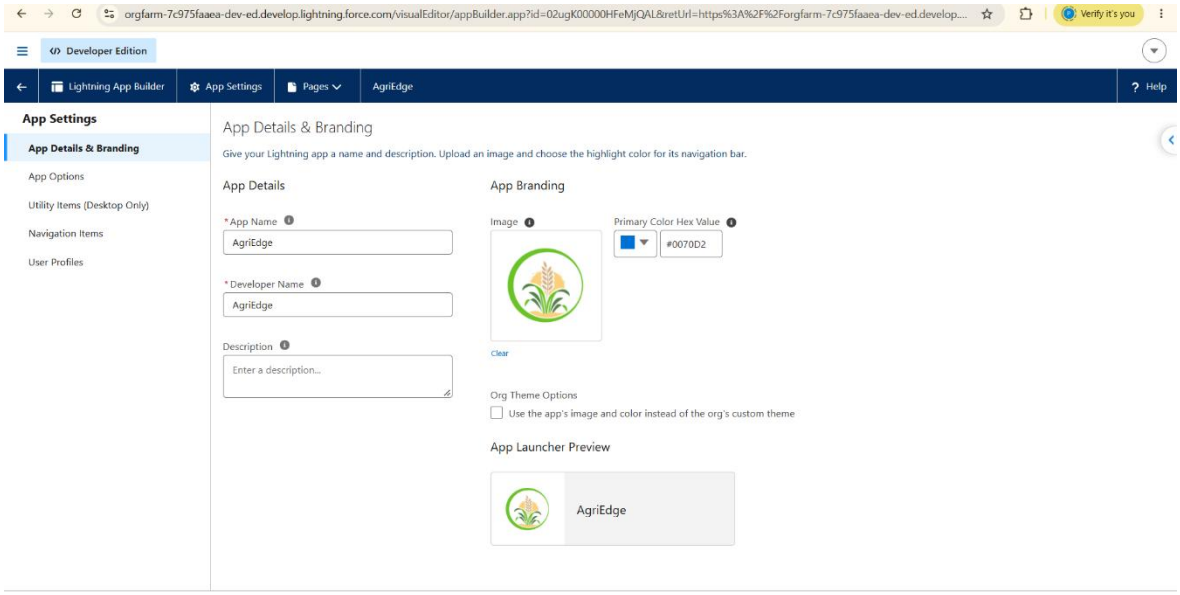
The main objectives of the project are:

1. **Automate Business Workflows:** Reduce manual work by automating order processing and follow-up tasks.
2. **Improve Operational Efficiency:** Automatically assign high-priority tasks to order owners.
3. **Maintain Data Integrity:** Ensure accurate order totals, payment statuses, and shipment information.
4. **Simplify Order Management:** Manage orders, order items, and shipments in one Salesforce application.
5. **Improve Business Visibility:** Provide reports and dashboards for monitoring order status and fulfillment.
6. **Ensure Code Reliability:** Develop and test Apex classes and triggers to support reliable automation.

KEY FEATURES

- Custom Salesforce application for order management.
- Automated follow-up task creation.

- Order status and payment status tracking.
- Order item management.
- Automatic shipment record creation after payment approval.
- Order total calculation through automation.
- Cleanup of order items when payment fails.
- Reports and dashboards for business monitoring.
- Security configuration and field history tracking.
- Apex unit testing.



PHASE 1: REQUIREMENT ANALYSIS AND PLANNING

UNDERSTANDING BUSINESS REQUIREMENTS

The project addresses the following business problems.

A. Manual Follow-Up Tasks

Sales representatives manually create tasks to follow up on newly submitted orders.

Proposed Solution: Automatically create a high-priority follow-up task whenever a new order is created.

B. Order Total Calculation Errors

Manually updating order totals when order items change can lead to incorrect financial information.

Proposed Solution: Implement automation to calculate and update order totals whenever order items are added, modified, or removed.

C. Delayed Shipment Processing

Payment verification and shipment creation may be handled separately, delaying order fulfillment.

Proposed Solution: Automatically create a shipment record when an order's payment status changes to Paid.

D. Inconsistent Order Information

Manual updates can cause inconsistencies between order records, order items, and shipment information.

Proposed Solution: Use Apex triggers and helper classes to automate related record operations and maintain data consistency.

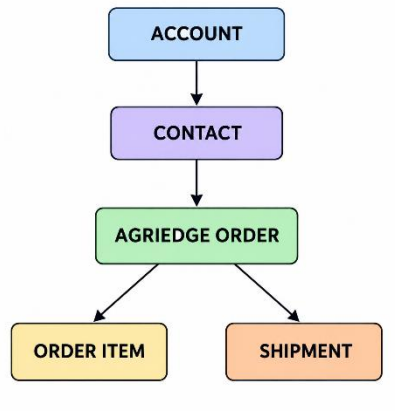
Project Scope

The project includes the following activities:

1. Creating custom Salesforce objects for orders, order items, and shipments.
2. Configuring relationships between Salesforce records.
3. Implementing Process Builder automation.
4. Developing Apex classes and triggers.
5. Creating custom application navigation and page layouts.
6. Configuring security and field history tracking.
7. Testing business logic using Apex test classes.
8. Creating reports and dashboards.
9. Preparing the application for deployment.

Data Model Design

Object Relationships



The application uses standard Salesforce objects and custom objects to manage order-related information.

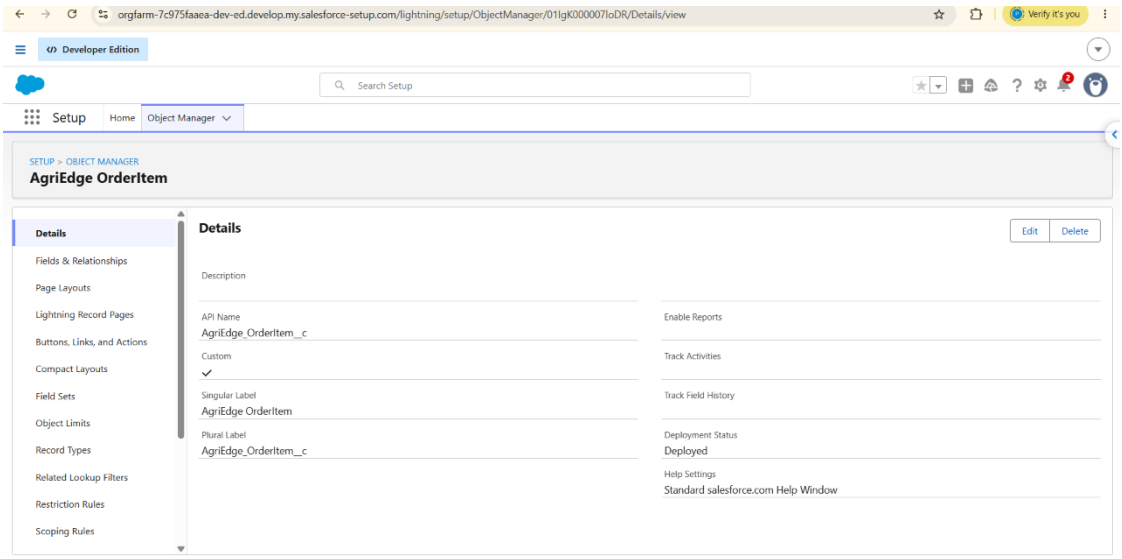
Relationship descriptions:

- **Account:** Stores customer or business information.
- **Contact:** Stores customer contact information.
- **Agri Edge Order:** Stores order details, order status, and payment status.
- **Agri Edge Order Item:** Stores individual products or items associated with an order.
- **Agri Edge Shipment:** Stores shipment and tracking information.

The exact relationship between Contact and Order, and the relationship types for Shipment, should match the configuration implemented in Salesforce.

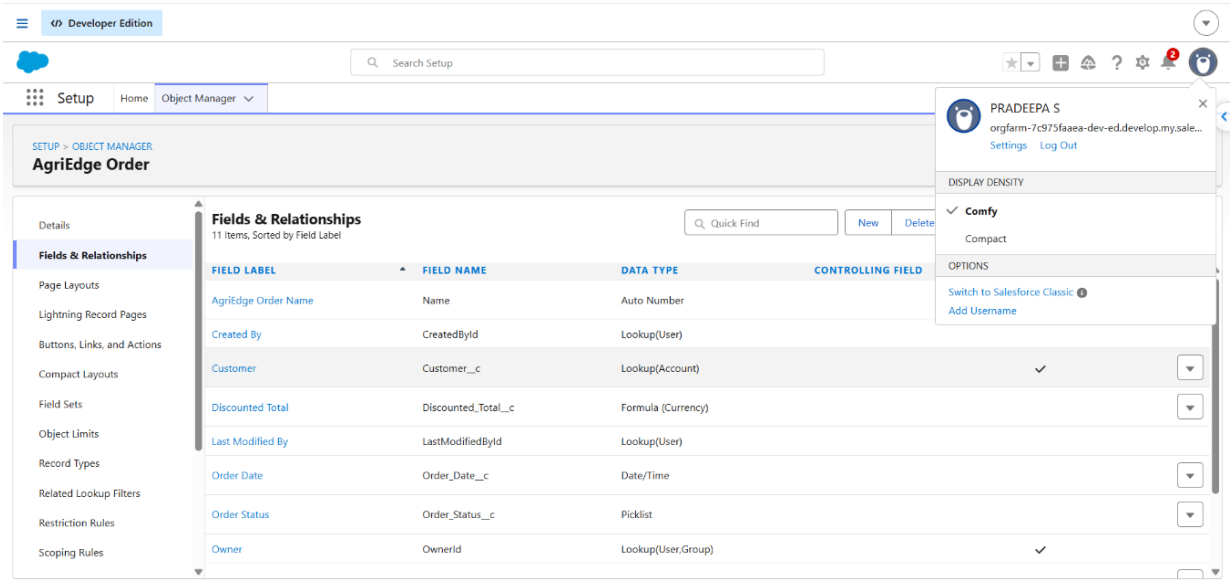
Data Dictionary

Object Name	API Name	Purpose
Agri Edge Order	AgriEdge_Order__c	Stores order information
Agri Edge Order Item	AgriEdge_OrderItem__c	Stores individual order items
Agri Edge Shipment	Agri Edge_Shipment__c	Stores shipment information
Account	Account	Stores customer or business information
Contact	Contact	Stores contact details



Important Fields:

Object	Field API Name	Data Type / Values
AgriEdge Order	Customer__c	Lookup or Master-Detail relationship, depending on configuration
AgriEdge Order	Order_Date__c	Date
AgriEdge Order	Order_Status__c	Picklist
AgriEdge Order	Payment_Status__c	Picklist
AgriEdge Order Item	Order__c	Master-Detail relationship
AgriEdge Order Item	Quantity__c	Number
AgriEdge Order Item	Unit_Price__c	Currency
AgriEdge Shipment	Order__c	Lookup or Master-Detail relationship
AgriEdge Shipment	Tracking_Number__c	Text
AgriEdge Shipment	Shipment_Status__c	Picklist



Order Status Values

- New
- Processing
- Shipped
- Delivered
- Cancelled

Payment Status Values

- Pending
- Paid
- Failed

Security Model Design

The following security configurations are planned for the application:

Profiles

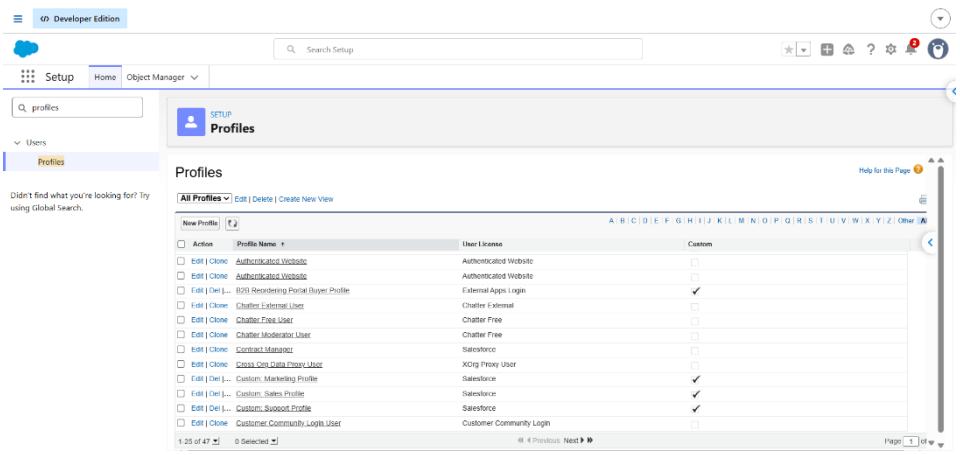
- System Administrator: Full administrative access.
- Standard User: Access based on assigned permissions.

Organization-Wide Defaults (OWD)

- Accounts: Public Read/Write, if appropriate for the organization's security requirements.
- Order Items: Controlled by Parent, where supported by the configured relationship.

Field-Level Security (FLS)

- Restrict editing of Payment_Status__c to authorized users.
- Restrict editing of Order_Status__c to authorized users.
- Configure object and field permissions according to user responsibilities.



PHASE 2 – SALESFORCE DEVELOPMENT AND CONFIGURATION

Development Environment

The project uses the following development tools and technologies.

Component	Technology
CRM Platform	Salesforce
Development Environment	Salesforce Developer Edition
Programming Language	Apex
Declarative Automation	Process Builder
Development IDE	Visual Studio Code
Salesforce Tools	Salesforce Extension Pack
Command-Line Tool	Salesforce CLI
Testing Environment	Salesforce Developer Console
Version Control	Git and GitHub

Process Builder – Order Task Creator

Purpose

The Order Task Creator automation creates a follow-up task whenever a new AgriEdge Order is created with the status New.

Configuration

- **Process Name:** Order Task Creator
- **Object:** AgriEdge_Order__c
- **Trigger:** When a record is created.
- **Criteria:** Order_Status__c equals New.
- **Action:** Invoke the OrderTaskCreator Apex method.

Expected Result

When a new order is created, Salesforce automatically creates a follow-up task assigned to the order owner.

The task includes:

- **Subject:** Follow up on New Order
- **Status:** Not Started
- **Priority:** High
- **Related Record:** AgriEdge Order

Apex Development

Apex is used to implement the backend business logic for order management.

The main components are:

1. OrderTaskCreator – Creates follow-up tasks.
2. AgriEdgeOrderTrigger – Detects relevant order updates.
3. AgriEdgeOrderShipmentHelper – Handles shipment creation and order item cleanup.

Invocable Apex Class – OrderTaskCreator

Purpose: Creates follow-up tasks for newly created orders.

File Name: OrderTaskCreator.cls

```
public class OrderTaskCreator {

    @InvocableMethod(
        label='Create Order Task'
        description='Creates follow-up task for new orders'
    )
    public static void createOrderTasks(List<Id> orderIds) {

        List<Task> tasksToInsert = new List<Task>();

        List<AgriEdge_Order__c> orders = [
            SELECT Id, OwnerId, Name
            FROM AgriEdge_Order__c
            WHERE Id IN :orderIds
```

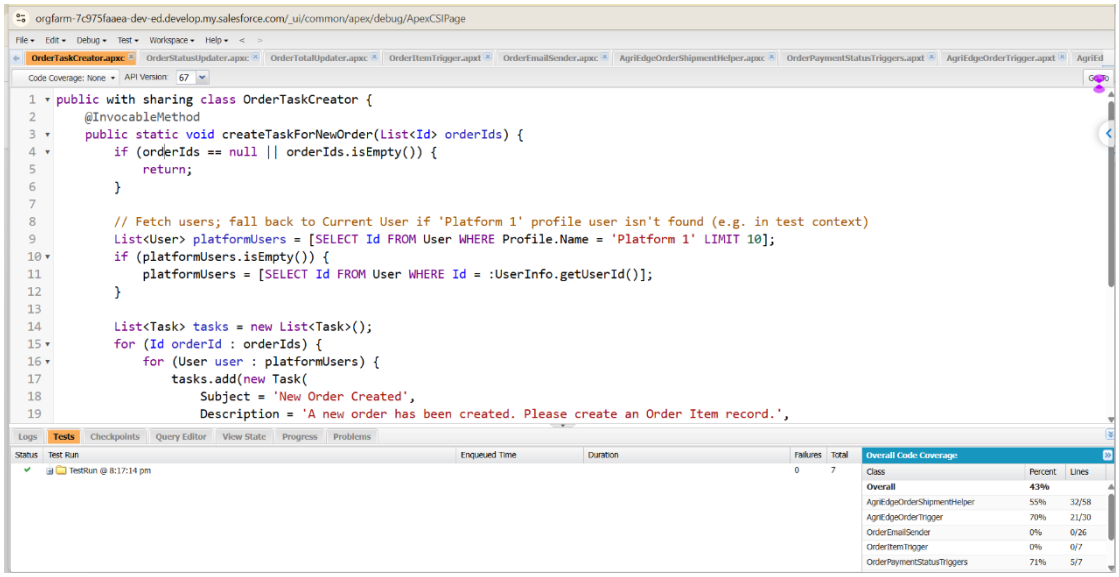
```
        ];

        for (AgriEdge_Order__c ord : orders) {

            Task t = new Task(
                Subject = 'Follow up on New Order',
                Status = 'Not Started',
                Priority = 'High',
                WhatId = ord.Id,
                OwnerId = ord.OwnerId
            );

            tasksToInsert.add(t);
        }

        if (!tasksToInsert.isEmpty()) {
            insert tasksToInsert;
        }
    }
}
```



Working Principle:

1. Receives order record IDs from the automation.
2. Retrieves the corresponding order records.
3. Creates a high-priority follow-up task for each order.
4. Inserts all generated tasks into Salesforce.

Apex Trigger – AgriEdgeOrderTrigger

Purpose: Detects changes to order status and payment status.

File Name: AgriEdgeOrderTrigger.trigger

```
trigger AgriEdgeOrderTrigger
on AgriEdge_Order__c (after update) {

    List<AgriEdge_Order__c> relevantOrders =
        new List<AgriEdge_Order__c>();

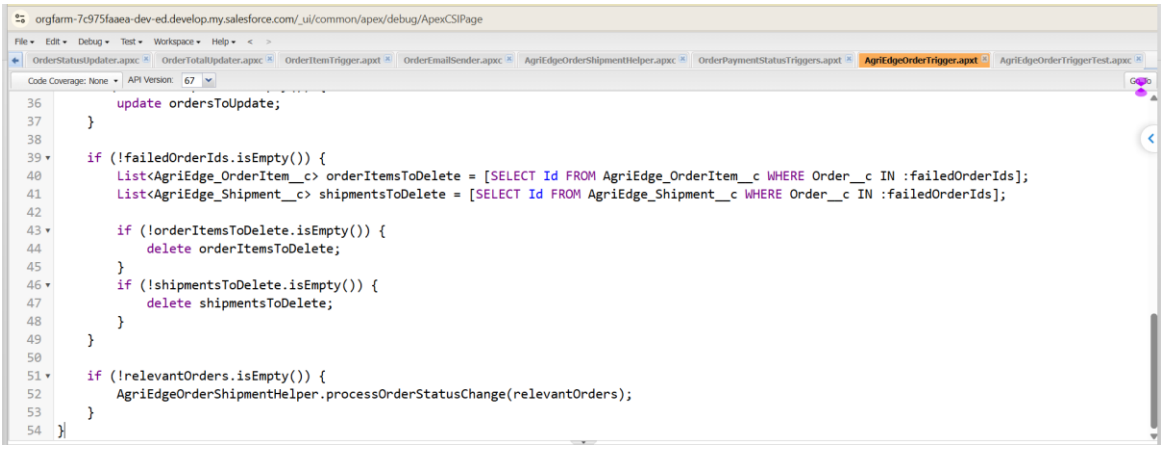
    for (AgriEdge_Order__c ord : Trigger.new) {

        AgriEdge_Order__c oldOrd =
            Trigger.oldMap.get(ord.Id);

        if (
            ord.Payment_Status__c != oldOrd.Payment_Status__c ||
            ord.Order_Status__c != oldOrd.Order_Status__c
        ) {
            relevantOrders.add(ord);
        }
    }

    if (!relevantOrders.isEmpty()) {
        AgriEdgeOrderShipmentHelper
```

```
        .processOrderStatusChange(relevantOrders);
    }
}
```



Working Principle:

1. Executes after an order record is updated.
2. Compares the updated record with its previous version.
3. Identifies changes to payment status or order status.
4. Passes relevant records to the helper class.

Helper Class – AgriEdgeOrderShipmentHelper

Purpose: Creates shipment records for paid orders and removes order items associated with failed payments.

File Name: AgriEdgeOrderShipmentHelper.cls

```
public class AgriEdgeOrderShipmentHelper {


    public static void processOrderStatusChange(
        List<AgriEdge_Order__c> orders
    ) {


        List<AgriEdge_Shipment__c> shipmentsToInsert =
            new List<AgriEdge_Shipment__c>();
    Set<Id> failedOrderIds = new Set<Id>(
```

```
for (AgriEdge_Order__c ord : orders) {

        if (ord.Payment_Status__c == 'Paid') {

            AgriEdge_Shipment__c ship =
                new AgriEdge_Shipment__c(
                    Order__c = ord.Id,
                    Tracking_Number__c =
                        'TRK-' +
                        String.valueOf(
                            Crypto.getRandomInteger()
                        ).substring(0, 6)
                );

            shipmentsToInsert.add(ship);

        } else if (ord.Payment_Status__c == 'Failed') {

            failedOrderIds.add(ord.Id);
        }
    }

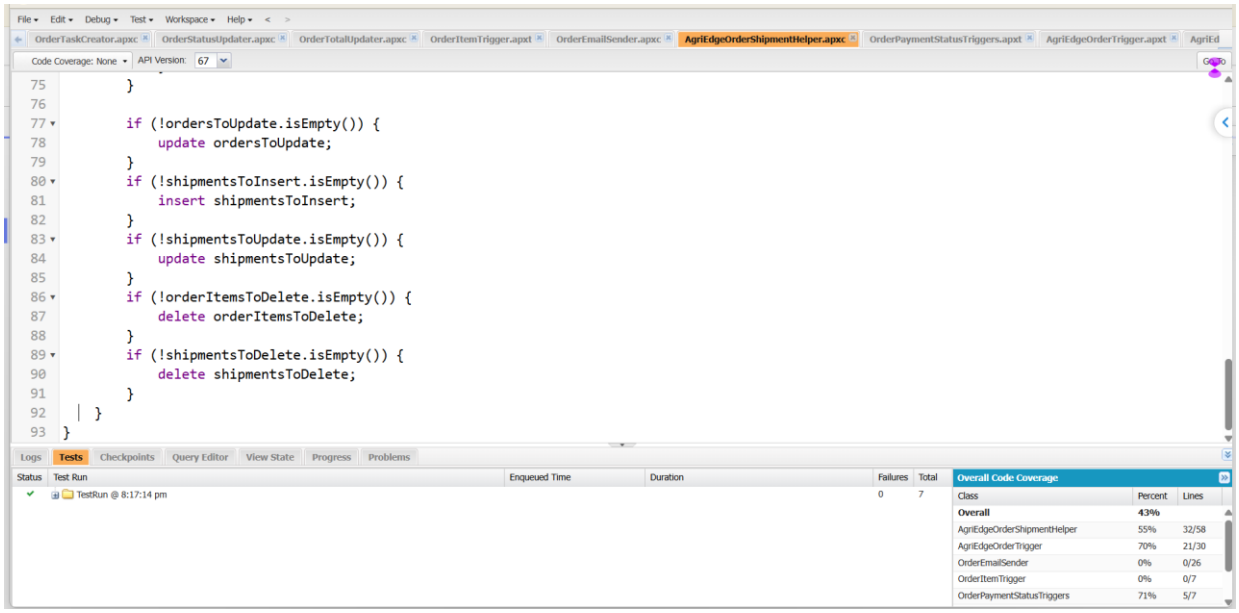
    if (!shipmentsToInsert.isEmpty()) {
        insert shipmentsToInsert;
    }

    if (!failedOrderIds.isEmpty()) {

        List<AgriEdge_OrderItem__c> itemsToDelete = [
            SELECT Id
```

```
            FROM AgriEdge_OrderItem__c
            WHERE Order__c IN :failedOrderIds
        ];

        if (!itemsToDelete.isEmpty()) {
            delete itemsToDelete;
        }
    }
  }
}
```



Working Principle:

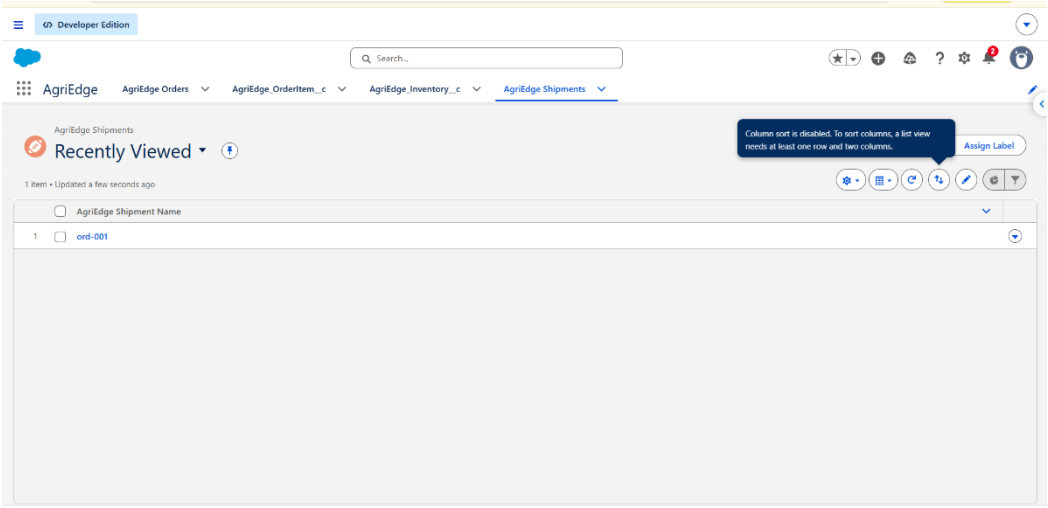
1. Receives order records that have relevant status changes.
2. Creates shipment records for orders whose payment status is Paid.
3. Generates a tracking number for each new shipment.
4. Identifies orders whose payment status is Failed.
5. Deletes associated order items for failed orders.

PHASE 3 – UI/UX DEVELOPMENT AND CUSTOMIZATION

Lightning Application Creation

A custom Salesforce Lightning application named **AgriEdge Order Management** is configured through Salesforce App Manager.

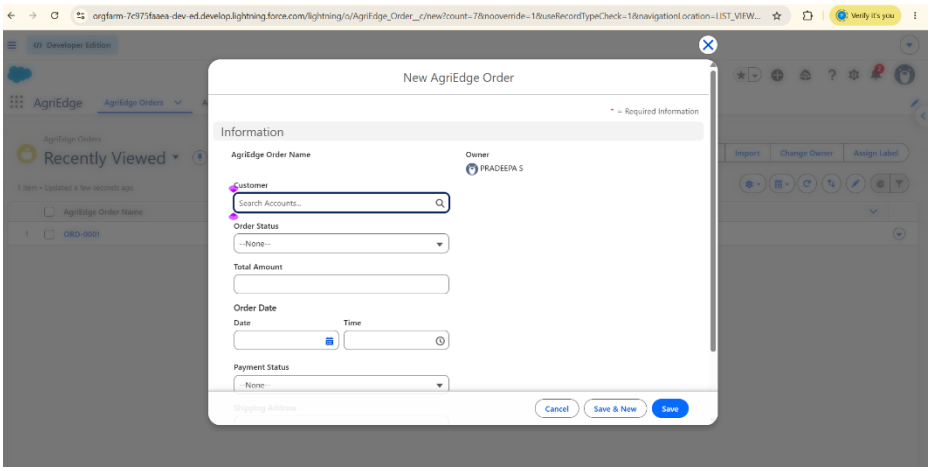
The application provides centralized access to order-related information.



Application Navigation

The application navigation menu includes:

1. Accounts
2. Contacts
3. AgriEdge Orders
4. AgriEdge Order Items
5. AgriEdge Shipments
6. Reports
7. Dashboards



Page Layout Customization

The AgriEdge Order page layout is organized into the following sections:

Order Information

- Order Name
- Customer
- Order Date
- Order Status

Financial Information

- Payment Status
- Order Items
- Order Total, if configured

Fulfillment Tracking

- Shipment Details
- Tracking Number
- Shipment Status

These sections make order information easier to view and manage.

PHASE 4 – DATA MIGRATION, TESTING AND SECURITY

Data Migration

Sample data is imported into Salesforce using the Data Import Wizard.

Data Import Process

1. Prepare customer and contact data in CSV format.
2. Open the Salesforce Data Import Wizard.
3. Select the appropriate Salesforce object.
4. Upload the CSV file.
5. Map CSV columns to Salesforce fields.
6. Validate the field mappings.
7. Import the records.
8. Verify the imported data.

Account, Contact, and product catalog records can be imported using the appropriate supported objects and field mappings.

Security Configuration

Field History Tracking

Field history tracking is configured for critical fields, including:

- Order_Status__c
- Payment_Status__c

This helps authorized users review changes to important order information.

Duplicate Management

Matching Rules and Duplicate Rules are configured for Account and Contact records to help identify and prevent duplicate customer information.

Access Control

Object permissions, field permissions, and record access are configured according to the responsibilities of administrators and standard users.

Unit Testing

Apex unit tests are used to verify the behavior of the project's backend logic.

The test class is identified as:

AgriEdgeOrderTriggerTest.cls

Test Cases

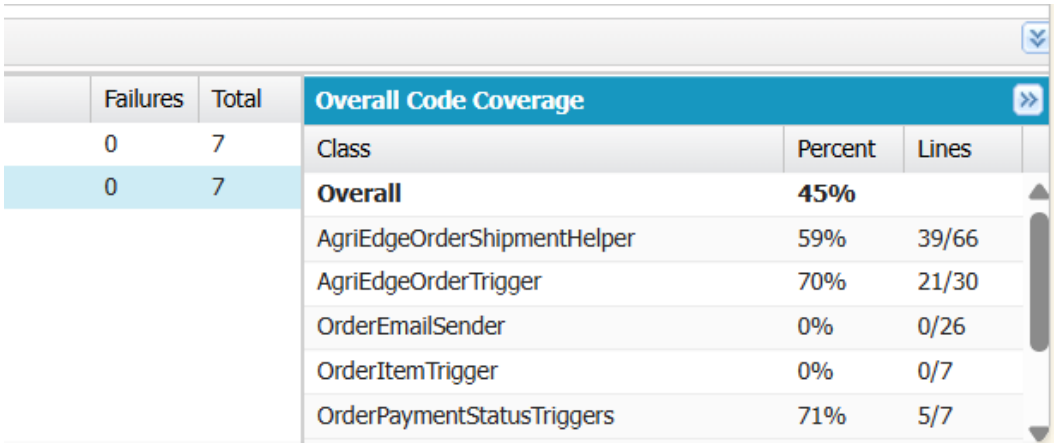
S. No.	Test Method	Functionality
1	testAgriEdgeOrderShipmentHelper	Tests shipment creation after payment completion
2	testAgriEdgeOrderTrigger	Tests trigger filtering and helper invocation
3	testOrderEmailSender	Tests confirmation email functionality, if implemented
4	testOrderItemTriggerAndTotalUpdater	Tests order total recalculation, if implemented
5	testOrderPaymentStatusTriggers	Tests payment status-related behavior
6	testOrderStatusUpdater	Tests order status updates, if implemented
7	testOrderTaskCreator	Tests follow-up task creation

Reported Test Results

The original project notes report the following test execution summary:

Metric	Reported Result
Total Test Methods	7
Passed Test Methods	7
Failed Test Methods	0
Test Pass Rate	100%

Verification Note: These figures should be included as final results only after confirming the actual Salesforce test execution results. A 100% test pass rate does not automatically mean 100% Apex code coverage. The organization's code coverage report must be checked separately.



Failures	Total
0	7
0	7

Overall Code Coverage

Class	Percent	Lines
Overall	**45%**	
AgriEdgeOrderShipmentHelper	59%	39/66
AgriEdgeOrderTrigger	70%	21/30
OrderEmailSender	0%	0/26
OrderItemTrigger	0%	0/7
OrderPaymentStatusTriggers	71%	5/7

PHASE 5 – DEPLOYMENT, MAINTENANCE AND TROUBLESHOOTING

Deployment Strategy

The application can be deployed using Salesforce CLI source deployment or Change Sets, depending on the deployment environment and requirements.

Deployment Steps

1. Complete development and configuration in the development environment.
2. Run Apex unit tests.
3. Verify code coverage and required metadata.
4. Validate the deployment in the target environment.
5. Deploy the required Apex classes, triggers, objects, fields, layouts, and automation.

6. Verify application functionality after deployment.
7. Monitor logs and resolve any deployment issues.

Target Environment: Salesforce sandbox or production environment, according to the project deployment plan.

System Maintenance

The following maintenance activities help keep the application reliable:

- Execute Apex tests after significant code changes.
- Monitor debug logs for exceptions.
- Review Salesforce governor limit usage.
- Check for duplicate shipment records.
- Validate payment and order status transitions.
- Review user permissions regularly.
- Verify reports and dashboards.
- Maintain source code using version control.

Troubleshooting

Issue	Possible Solution
Follow-up task is not created	Check Process Builder activation, criteria, and Apex invocation
Shipment is not created	Verify payment status, trigger execution, and helper class
Duplicate shipments are created	Implement duplicate prevention logic
Order total is incorrect	Verify order item automation and calculation logic
Users cannot edit a field	Review field-level security and object permissions
Apex tests fail	Review test data, assertions, validation rules, and exception messages
Deployment fails	Check metadata dependencies, permissions, tests, and deployment errors

FUTURE ENHANCEMENTS

The project can be improved with the following additional features.

Einstein AI Integration

Integrate predictive analytics to forecast order volumes and identify potential fulfillment delays.

Lightning Web Component Order Portal

Develop a customer-facing Lightning Web Component that allows customers to view order details and track shipments.

Automated WhatsApp Notifications

Integrate an appropriate Salesforce messaging service or supported third-party provider to send order confirmations, payment updates, and shipment notifications.

Advanced Analytics

Develop additional reports and dashboards to analyze sales performance, order processing time, shipment delays, and customer purchasing patterns.

Improved Shipment Management

Add unique tracking number generation, duplicate shipment prevention, and automated shipment status updates.

CONCLUSION

The Agri Edge Order Management CRM is designed to simplify agricultural order management using Salesforce platform capabilities.

The application combines custom objects, Apex classes, triggers, declarative automation, security configurations, and reports to support the order processing lifecycle.

Its key features include automated follow-up task creation, payment status tracking, shipment record generation, and order item management.

By reducing manual activities and centralizing order information, the system aims to improve operational efficiency, data consistency, and business visibility.

The project also provides opportunities for future enhancements through AI-based forecasting, customer-facing shipment tracking, and automated notifications.

Overall, Agri Edge Order Management CRM demonstrates how Salesforce development and automation can be applied to address practical order management challenges in the agricultural supply chain.